

Rating Watchlist

Technical report — application structure, module reference and operating notes

Purpose	Collect the latest credit rating for a fixed watchlist of entities from seven SEBI-registered rating agencies, and flag where NaBFID appears as a lender
Scope	36 entities: trusts, InvITs, municipal corporations, foundations and project SPVs — the segment a commercial rating vendor does not cover
Agencies	CARE, CRISIL, ICRA, India Ratings, Brickwork, Infomerics, Acuite
Output	Excel workbook: Ratings sheet (15 columns), QA sheet, Summary sheet
Codebase	5,993 lines of Python across 22 modules, plus 456 lines of tests
Runtime	Python 3.13, Windows. Nine third-party packages, all from PyPI
External services	None. No paid API, no API key, no account, no login
Report date	19 August 2026

1. What the application does

Credit ratings for infrastructure borrowers are published by rating agencies as press releases and rationale documents on their own websites. There is no common feed and no shared identifier: each agency uses its own spelling of a borrower's name, its own document format, and its own search interface. Collecting a portfolio's ratings has therefore been a manual exercise — opening seven websites per entity, reading each document, and copying figures into a spreadsheet.

This application automates that. For each entity on a persistent watchlist it searches all seven agencies, locates the most recent rating document, extracts every rated instrument with its rating, amount and date, reads the lender annexure to determine whether NaBFID is a lender and for how much, and writes the result to Excel.

Two design rules shape everything else. **First, a rating older than fifteen months is reported blank rather than stale** — an expired opinion is worse than no opinion. **Second, where the application cannot be certain which agency entity corresponds to a watchlist entry, it produces no row rather than a guess**, and records the ambiguity in a QA sheet. Both rules trade coverage for correctness, deliberately.

2. How a run works

One run processes every active entity against every enabled agency. The seven agencies are worked in parallel, one thread each, while requests within an agency stay serial and throttled so no single site sees a burst.

```
for each entity on the watchlist:
    for each of the 7 agencies (in parallel):

        1. candidate_names()    ask the agency's index who it has
        2. pick_company()       score candidates against the full entity name
                                -> no confident winner? emit a QA row, no rating
```

3. search() / latest() find the newest rating document
4. parse() extract one record per rated instrument
5. lenders() read the lender annexure
6. apply_nabfid() stamp NaBFID exposure onto every record
7. age filter drop anything older than 15 months

write Ratings + QA + Summary to one Excel workbook

Every step that produces nothing writes a line to the QA sheet explaining why, so a blank cell in the output can always be accounted for.

3. Directory structure

```
final_rating_scrapper/
  INSTALL.bat          one-time setup (offline install + self-check)
  START.bat            launches the web application
  README.md            installation and usage
  DEPLOYMENT.md        for IT: hostnames, packages, licences
  TROUBLESHOOTING.md   failure modes and remedies
  requirements.txt      nine packages, minimum versions
  requirements-lock.txt exact versions shipped in wheels/
  wheels/              33 packages for installation without internet
  run_watchlist.py      command-line entry point

data/                  persistent state
  entities.db           the watchlist
  resolutions.db        confirmed identities and pinned URLs
  cache/               cached HTTP responses (created on first run)
  runs/                Excel files produced by the web application

src/rating_scraper/    the library
  models.py normalize.py entities.py resolutions.py pipeline.py
  http_client.py pdf_utils.py excel_export.py cli.py
  agencies/            one adapter per agency, plus base.py

webapp/               app.py, jobs.py, static/
tests/                three suites, all run offline
tools/                diagnostics
```

4. Core modules

Nine modules form the library. They are listed here in dependency order — models and normalize are depended upon by everything else; pipeline depends on all of them.

Module	Lines	Responsibility
models.py	246	The data structures every other module exchanges: Company (a watchlist entry), Document (a located rating release), RatingRecord (one output row), LenderLine (one row of a lender annexure). Also defines OUTPUT_COLUMNS, the single definition of the Excel column order, and search_terms() which generates the query variants sent to an agency.
normalize.py	564	The correctness core. Company-name matching, and the parsing of ratings, amounts, dates and instrument categories out of free text. Described in detail in section 5.

Module	Lines	Responsibility
entities.py	212	The watchlist store. Adds, deactivates and deduplicates entities, and infers an entity type (InvIT, Trust, Municipal Body, Foundation, Society) from the name. Deduplication is by canonical name, so case and acronym variants of one entity cannot be added twice.
resolutions.py	211	Two kinds of permanent decision: a confirmed mapping from a watchlist entity to an agency's own entity name, and a pinned document URL for ratings an agency publishes but omits from its search. Both survive restarts and make later runs deterministic.
pipeline.py	413	Orchestration. Runs entities across agencies with one thread per agency, applies the fifteen-month rule, reduces to the latest rating per instrument category, and assembles the QA sheet.
http_client.py	230	Every outbound request. Per-host rate limiting, retry with backoff on 429/5xx, and a content-addressed disk cache (SQLite index plus blob files) that makes re-runs cheap without hiding new documents.
pdf_utils.py	141	PDF text and table extraction, used by the agencies that publish PDFs rather than HTML. Combines two engines: PyMuPDF for text and links, pdfplumber for table grids.
excel_export.py	115	Writes the workbook: Ratings, QA and Summary sheets, with column widths, header styling and frozen panes.
cli.py	178	The older per-company command line, retained for ad-hoc lookups outside the watchlist.

5. normalize.py — name matching and text parsing

This module carries the highest risk in the system. If it matches two different companies, a rating is attached to the wrong borrower and nothing downstream will notice. Every rule in it exists because a real pair of names defeated the previous version.

Name matching

`name_score(a, b)` returns 0–100. A score of 88 or above is accepted as the same company; two candidates within two points of each other are treated as a tie and rejected as ambiguous rather than resolved arbitrarily.

Rule	Why it exists
Canonical form	Case, punctuation, underscores and legal suffixes (Private Limited, Pvt. Ltd., LLP) are folded away before comparison.
Acronym suffix removal	'Surat Municipal Corporation (SMC)' equals 'Surat Municipal Corporation'. Only true acronyms of the name are stripped: '(Bawana)' is kept, because there the bracket is the identity.
Closed-up compounds	Identical letters with different word breaks are the same entity. CRISIL indexes 'Oriental Infratrust' for 'Oriental Infra Trust'; compared as tokens these score 49.
Agency typo folding	Agencies misspell their own entities. India Ratings indexes 'CORPORTION'. A stray letter otherwise makes a name look like a different company.
Numeric identity	Any disagreement in digits caps the score at 50. 'Huoban Energy 1' and 'Huoban Energy 11' are separate borrowers with separate ratings.

Rule	Why it exists
Distinctive-token overlap	Two names sharing only trade vocabulary are not the same company. 'Suriya Spinning Mills' and 'S.S. Spinning Mills' agree on 'spinning mills' and nothing else.
Subset cap	Where one name is wholly contained in the other, the score is capped below the acceptance bar. 'BNP Paribas' against 'BNP Paribas India Solutions Private Limited' is a bank against its captive subsidiary; both scored 97 before this rule.
Entity-type awareness	For trusts, InvITs and civic bodies the words infrastructure, energy, education and national carry identity rather than noise, so they are exempted from the generic-word list that applies to ordinary companies.

Text parsing

Function	Purpose
<code>split_rating()</code>	Separates a rating from its outlook across seven agency house styles: 'CARE AAA; Stable', '[ICRA]AAA (Stable)', 'IND AA+/Stable', 'Crisil AAA/Stable', 'BWR AA-', 'IVR A+/Stable'.
<code>extract_action()</code>	The rating action — Assigned, Reaffirmed, Upgraded, Downgraded, Withdrawn — which agencies state in prose or in brackets.
<code>classify_instrument()</code>	Buckets an instrument into a category: Long Term Bank Facilities, Debt Instrument, Commercial Paper, Issuer Rating, InvIT Units and others.
<code>parse_amount()</code>	A rated amount from text, tolerating commas, currency symbols and '(Enhanced from ...)' annotations.
<code>detect_unit()</code>	Whether figures are in crore or million.
<code>parse_date()</code>	Publication dates across the formats the seven agencies use.
<code>is_nabfid()</code>	Whether a lender name is NaBFID, allowing for the abbreviation, the full name and the spacing variants that appear in annexures.

6. Agency adapters

Each agency has one adapter implementing a common contract defined in `base.py`. The pipeline calls only four methods — `search`, `latest`, `parse`, `lenders` — so an eighth agency can be added without touching anything else.

Adapter	Lines	Source and access method
<code>base.py</code>	268	The shared contract, plus the matching and ambiguity logic every adapter inherits: candidate de-duplication, scoring against the full entity name rather than the query, and <code>AmbiguousMatchError</code> .
<code>crisil.py</code>	772	Two JSON datasets behind an Adobe AEM component. The credit-ratings list (<code>cmd=CR</code>) supplies instrument-level rows; the rating-rationale list (<code>cmd=RR</code>) is larger and is the only route to entities missing from the first. Rationales are HTML and carry the lender annexure inline.
<code>infomerics.py</code>	537	A Strapi CMS. A company index and a file index are downloaded and matched by filename; rating documents are PDFs on Azure blob storage under a separate hostname.

Adapter	Lines	Source and access method
care.py	382	A search endpoint returning HTML, then a PDF per press release. Handles pooled 'Restricted Group' updates, where one document covers many SPVs and each member links to its own release.
icra.py	361	A rationale listing plus per-document pages; lender details arrive from a separate endpoint.
indiaratings.py	351	A single JSON endpoint resolves the issuer and lists its press releases together. Surveillance notices ('Update on ...') carry no rating table and are stepped over.
brickwork.py	330	An ASP.NET search form; press releases are PDFs.
acuite.py	282	A JSON autocomplete for entity names, and an HTML endpoint for ratings whose CSRF token must travel as a header. The lender annexure is on a separate per-action detail page.

Why the adapters differ so much in size

The line counts reflect how cooperative each agency's site is, not how important the agency is. CRISIL is the largest because it required four separate extraction paths: the instrument listing, the document's own rating-action table, withdrawal notices in a third layout, and the lender annexure — each of which is the only source of a rating for some entities. Acuite is the smallest because its data is well structured once the header requirement is understood.

7. Web application

File	Lines	Responsibility
webapp/app.py	226	Flask API and page. Endpoints for listing, adding, uploading, editing and removing watchlist entities; starting a run; polling its progress; and downloading the result. Binds to 127.0.0.1 only.
webapp/jobs.py	140	Background job runner. A full run takes minutes, far longer than a browser will wait, so runs execute on a worker thread and the page polls for status. Results are written to disk so a download survives a page reload.
static/index.html	—	One page: add box, upload, entity list, progress, result.
static/style.css	—	Styling, with light and dark palettes selected from the operating system setting.
static/app.js	—	Loads the watchlist, manages selection and filtering, starts a run and polls it, renders the result.

Static files are served with `Cache-Control: no-store`. The page lives at a fixed address, so a browser that had visited an earlier build would otherwise serve stale JavaScript against fresh HTML — producing an empty entity list with no error to explain it.

8. Data stores

Store	Contents	Back up?
data/entities.db	The watchlist: name, inferred type, active flag, notes. Deduplicated by canonical name.	Yes — 16 KB

Store	Contents	Back up?
data/resolutions.db	Confirmed entity identities per agency, and pinned document URLs. Records human decisions that would otherwise be repeated.	Yes — 135 KB
data/cache/	Cached HTTP responses, content-addressed. Search listings expire after six hours; rating documents are immutable and kept indefinitely.	No — rebuilds
data/runs/	Excel files produced by the web application.	Optional

The split matters operationally: the two small databases are the only irreplaceable state. The cache can reach several hundred megabytes and can be deleted at any time; doing so only makes the next run slower.

9. Output format

The Ratings sheet carries one row per rated instrument, in fifteen columns:

```
Borrower Name | CIN | Open Charges | Rating Date | Rating Agency Name |
Instrument Category | Instrument Details | Amount | Rating | Development |
Outlook | Online url | NaBFID Name | Loan Amount | Unit
```

Column	Note
NaBFID Name	Yes or No, read from the agency's lender annexure rather than from body text, so a passing mention of NaBFID cannot produce a false Yes.
Loan Amount	NaBFID's exposure, summed across every facility it lends on. A single lender routinely appears on several annexure lines.
Online url	The exact document the row was extracted from — the starting point whenever a figure needs checking.
Rating Date	Taken from the document body where the agency prints it, which is more reliable than any date inferred from a filename.

The QA sheet explains every entity that produced no rating: `not_rated`, `outside_age_window` (with the date that was rejected), `needs_confirmation` (with the candidate names), `parse_failed`, `search_failed`. A blank is therefore always explainable.

10. Tests and diagnostics

File	Lines	What it protects
tests/test_extreme_names.py	161	38 name cases drawn from real agency data: closed-up compounds, separators, suffix forms, acronym tails, agency typos, plurals. Asserts three things — that true variants match, that look-alikes stay apart, and that an entity name generates a query capable of retrieving the agency's spelling. Also checks that each adapter has exactly one canonical name.
tests/test_normalize.py	135	45 assertions over rating, amount, date and instrument parsing across the seven house styles.
tests/test_matching.py	160	38 checks on the matching thresholds and the ambiguity rules.
tools/check_entity_names.py	121	Asks all seven agencies what they call each watchlist entity, and reports where our spelling differs.
tools/sweep_single_agency.py	115	Re-probes entities currently held from one agency only, to find ratings lost to a spelling difference rather than genuinely absent.

All three test suites run entirely offline. This is deliberate and operationally important: on a restricted machine they separate a broken installation from a blocked network, which look identical from the outside and are resolved by different people. INSTALL.bat runs them automatically at the end of setup.

11. Known limits

Limit	Detail
Coverage is 35 of 36 entities	The remaining entity, Vidya Vikas Education Trust, is rated — but every rating is older than fifteen months, so the age rule blanks it. This is the rule working, not a failure.
Some ratings are absent from an agency's own search	CRISIL hosts rationale documents for entities its search index does not list. Four such cases are known. The document URL can be pinned once, after which the rating is collected normally.
Location and Open Charges are not collected	Rating agencies do not publish them; they come from MCA registry data and would need a separate source.
Most of these entities have no CIN	Trusts, InvITs and municipal bodies are not registered under the Companies Act. A confirmed name is therefore the only identity anchor available, which is why confirmations are stored permanently.
Only CRISIL has been back-tested in depth	Every defect found so far came from comparing output against source documents. That exercise has been completed for CRISIL and not for the other six agencies. Spot-checking amounts against the linked documents remains the highest-value verification available.
Strict mode has no interface	The pipeline supports collecting only against human-confirmed identities, which would guarantee no wrong rows. The API accepts it; there is not yet a screen for working through the confirmation queue.

12. Security and operating profile

Property	Position
Credentials	None. No API key, account, login or password anywhere in the source. The only token handled is a public per-session CSRF token issued by one agency's own page to any visitor; it is never stored.
Data egress	None. Public web pages in; Excel and local SQLite out. No telemetry, no analytics, no upload.
Network exposure	None inbound. The web application binds to 127.0.0.1 and is not reachable from the network.
Privilege	Runs as the ordinary user. Writes only inside its own folder. Installs no service and no scheduled task.
Access method	Reads the same public pages a browser would, throttled to roughly one request per second per host. Bypasses no paywall, login or CAPTCHA, and does not present itself as a search-engine crawler.
Outbound requirement	HTTPS to nine hostnames, listed in DEPLOYMENT.md. Two are easily missed: the Azure blob host serving Infomerics PDFs, and www.crisil.com, which is distinct from www.crisilratings.com.

Property	Position
Licences	Eight of the nine packages are MIT or BSD. PyMuPDF is AGPL — the one item warranting a licence review, and replaceable with additional work if refused.

Developed by Nikhil Pandit